

Andleeb Shehzadi

Sap id : 54997

Architecture Design Document: Among Us

1. Game Overview :

Among Us is a multiplayer social deduction game supporting 4 to 15 players. Players are split into two teams: Crewmates and Impostors. The core gameplay loop revolves around the following mechanics:

- **Matchmaking & Lobbies:** Players join public or private rooms using unique room codes generated by the server.
- **Task Completion:** Crewmates navigate the map to complete various mini-games (tasks) to fill a collective progress bar.
- **Sabotage & Elimination:** Impostors covertly sabotage map systems (like turning off lights or depleting oxygen) and eliminate Crewmates without being caught.
- **Social Consensus (Voting):** When a dead body is reported or an emergency button is pressed, the game transitions into a discussion phase where players use text or voice chat to debate and vote to eject suspected Impostors.

2. High-Level Architecture:

To ensure scalability, security, and real-time synchronization, this design utilizes an **Authoritative Server Architecture**. All critical gameplay logic, player states, and interactions are processed and validated on the server side to prevent client-side cheating and hacking.

The system is divided into several highly decoupled sub-systems:

- **Matchmaking & Lobby System:** Responsible for managing rooms, handling player connections, generating lobby codes, and synchronizing host-customized game rules (such as player speed, kill cooldowns, and number of impostors).
- **Player & Role System:** Manages individual player data, including unique network IDs, current spatial positions, assigned roles (Crewmate or Impostor), and vital status (Alive or Ghost).
- **Game State Management System:** Controls the overarching game flow and global states, ensuring strict transitions between phases (Lobby, Gameplay, Discussion/Voting, and End Game).

- **Task & Sabotage System:** Server tracks the real-time completion percentage of assigned tasks and monitors the timers, resolution status, and cooldowns of Impostor sabotages.
- **Combat & Interaction System:** Processes and validates proximity-based interaction requests sent by clients, such as "Kill", "Report Body", or "Use/Task".
- **Network Synchronization System:** Handles low-latency, real-time replication of player transforms (position and rotation) and dynamic game events across all connected clients.

3. Code Structure & System Responsibilities:

This architecture adopts a **Component-Based Design** combined with standard software engineering design patterns to keep the codebase modular, testable, and maintainable.

Core Classes & Components:

NetworkManager (Singleton Pattern):

- **Responsibility:** Establishes and maintains the network socket connection between the client and the server. It handles low-level serialization and deserialization of incoming and outgoing data packets.
- **Key Methods:** ConnectToServer(), Disconnect(), SendRPC(string methodName, object[] args).
- **Responsibility:** Manages the active game loop on the server. It ensures that inputs and systems behave correctly depending on the current phase of the game.

Key Fields: CurrentState GameStateManager (State Pattern):

- (Enum: LOBBY, PLAYING, VOTING, END_GAME).
- **Key Methods:** StartMatch(), TriggerEmergencyMeeting(), CheckWinConditions().

PlayerController (Entity Class)

- **Responsibility:** Acts as the base entity for every player in the game. It listens to local inputs on the client side and updates states on the server.
- **Attached Components:**
 - NetworkTransform: Synchronizes movement data across the network using interpolation to prevent visual jittering.
 - PlayerIdentity: Stores local variables like IsImpostor and IsAlive.

- **Key Methods:** Move(Vector2 direction), Interact(), CmdKillPlayer(int targetID).

VoteManager (Class)

- **Responsibility:** Active only during the voting phase. It registers votes anonymously, keeps track of skipped votes, and determines which player receives the highest tally.
- **Key Methods:** CastVote(int voterID, int targetID), TallyVotes(), EjectPlayer().

System Communication & Data Flow

Systems communicate asynchronously using **Observer Pattern (Events)** and **Remote Procedure Calls (RPCs)**:

1. The Elimination Flow (Kill Interaction):

- The client controlling an Impostor presses the "Kill" button. This triggers CmdKillPlayer(targetID) from the client's PlayerController to the Server.
- The server-side CombatSystem intercepts this request and validates it: checks if the sender is actually an Impostor, if the target is an alive Crewmate, and if the physical distance between them is within the allowed radius.
- If valid, the server changes the target's PlayerIdentity.IsAlive to false, spawns a dead body object at that coordinate, and broadcasts an RPC (RpcOnPlayerDied(targetID)) to update all clients visually.

2. The Progression Flow (Task Completion):

- A client successfully finishes a mini-game UI interface. The local TaskComponent fires an event.
- An RPC notification is sent to the server's TaskSystem. The server increments the global task counter, evaluates if the total tasks completed match the victory threshold, and syncs the updated progress bar UI to all connected players.

4. Design Decisions & Technical Reasoning

The selection of this specific architectural design is driven by concrete technical requirements for a modern multiplayer social deduction game:

- **Why Authoritative Server?** In a game reliant on hidden information (who the Impostor is) and wall boundaries, a peer-to-peer or client-hosted structure is highly vulnerable to memory hacking. By running all validation logic strictly on the server, clients can never spoof their position coordinates, bypass kill cooldown timers, or force-reveal player roles.

- **Why Component-Based Structure?** Relying on deep inheritance hierarchies (e.g., creating a separate GhostPlayer class that inherits from DeadPlayer which inherits from Player) leads to rigid code maintenance issues. Using components allows the runtime addition or modification of behaviors. For instance, when a player dies, the codebase simply toggles off their physics collider component and enables a GhostFlyComponent on the same object.
- **Why State Pattern for Phase Management?** The mechanics of Among Us change drastically between phases. In the gameplay phase, players move around freely but cannot chat. In the voting phase, player movement is frozen, and the UI text/voice channels open up. Managing this with massive if-else blocks creates spaghetti code. A State Pattern cleanly isolates the execution logic for each phase, preventing overlapping state bugs.
- **Why UDP Protocol over TCP for Syncing?** Real-time movement requires fast updates. TCP's overhead with error-checking and packet retransmission can introduce sudden latency spikes (lag). Utilizing UDP for position syncing ensures the latest spatial data is processed immediately, while minor packet losses are safely ignored because the next upcoming packet will instantly override the missed position.